

Laboratory Exercise No. 1

Aim: Prepare a complete Software Requirement Specification (SRS) for an assigned system (e.g., Library Management System/Online Food Delivery Application).

Theory:

- A Software Requirement Specification (SRS) is a comprehensive, structured document acting as a blueprint for software development, outlining the functional, non-functional, and behavioral requirements of a system.
- It serves as an agreement between stakeholders and developers, reducing rework by defining user needs, system features, and performance criteria.
- A software requirements specification is a detailed description of the intended purpose, features, functionality and other aspects of a software application.
- It serves as a guide and foundational reference for the development team to follow during the building of the software.
- A good software requirements specification will aid in reducing errors and rework later on in the development life cycle.
- The software requirement specification document captures what the software should do from the user's perspective.
- It should be written in clear language understandable to both technical and non-technical stakeholders.
- The document acts as a contract between the project sponsor or client and the development team regarding the conditions that the final software product must meet.
- It acts as a contract between client and developer and is based on the IEEE 830 / ISO/IEC/IEEE 29148 standard structure.

Chapters (Description):

1. Introduction

1.1 Purpose

- Explains why the SRS document is written
- Identifies the intended audience (developers, testers, managers, users)

1.2 Scope

- Describes the software system, its objectives, and benefits
- Defines what the system will and will not do

1.3 Definitions, Acronyms, and Abbreviations

- Lists technical terms, abbreviations, and domain-specific words

1.4 References

- Lists related documents, standards, books, or URLs

1.5 Overview

- Summary of the remaining sections of the SRS

2. Overall Description

2.1 Product Perspective

- Describes how the software fits into a larger system
- Mentions whether the product is standalone or part of a system

2.2 Product Functions

- High-level list of system features and capabilities

2.3 User Classes and Characteristics

- Identifies different types of users
- Describes their skills, responsibilities, and access levels

2.4 Operating Environment

- Hardware platforms
- Operating systems
- Databases, browsers, networks

2.5 Design and Implementation Constraints

- Hardware, software, legal, and policy constraints
- Programming language or database restrictions

2.6 User Documentation

- Manuals, help files, tutorials, FAQs

2.7 Assumptions and Dependencies

- Factors assumed to be true
- External systems or services the software depends on

3. Functional Requirements

- This section defines what the system must do.

3.1 System Features

For each feature:

- Description
- Priority
- Inputs
- Processing
- Outputs

Example:

- User Authentication
- Data Validation
- Report Generation

3.2 Use Case Specifications (optional but recommended)

- Use case name
- Actors
- Pre-conditions
- Main flow
- Alternate flows
- Post-conditions

4. External Interface Requirements

4.1 User Interfaces

- Screen layouts
- Navigation flow
- Accessibility requirements

4.2 Hardware Interfaces

- Hardware devices connected to the system

4.3 Software Interfaces

- Databases
- Operating systems

- External applications/APIs

4.4 Communication Interfaces

- Protocols (HTTP, HTTPS, TCP/IP)
- Data formats (JSON, XML)

5. Non-Functional Requirements

- Defines how well the system performs.

5.1 Performance Requirements

- Response time
- Throughput
- Load handling

5.2 Security Requirements

- Authentication and authorization
- Data encryption
- Audit trails

5.3 Reliability Requirements

- Availability
- Fault tolerance
- Recovery time

5.4 Usability Requirements

- Ease of use
- Learning curve
- User interface consistency

5.5 Maintainability

- Modularity
- Code documentation
- Ease of updates

5.6 Scalability

- Handling increased users, data, or transactions

5.7 Portability

- Ability to run on different platforms

6. System Models

6.1 Use Case Diagrams

- Graphical representation of user interactions

6.2 Data Flow Diagrams (DFD)

- Flow of data through the system

6.3 Sequence Diagrams

- Interaction sequence among system components

7. Data Requirements

7.1 Data Description

- Data types
- Data constraints
- Validation rules

7.2 Database Requirements

- Tables
- Keys

8. Quality Attributes

- Correctness
- Efficiency
- Flexibility
- Reusability
- Testability

9. Constraints and Risks

- Technical constraints
- Project risks
- Security and compliance risks

10. Acceptance Criteria

- Conditions under which the system will be accepted
- Validation and verification measures

11. Appendices

- Glossary
- Definitions of terms

Sample Software Requirement Specification (SRS) Document

Online Food Delivery Application

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of the Online Food Delivery Application. The purpose of this document is to provide a clear understanding of system behavior to developers, testers, project managers, and stakeholders. It serves as a contract between client and development team.

Intended Audience:

- Developers – system implementation
- Testers – test case creation
- Project Managers – planning & tracking
- Clients/Users – understanding system features

1.2 Scope

The Online Food Delivery Application allows customers to search restaurants, order food, make payments, and track deliveries in real time.

Objectives

- Provide convenient food ordering platform
- Reduce waiting time
- Enable digital payment
- Improve restaurant reach and efficiency

Included Features

- User registration and login
- Restaurant browsing
- Add to cart and order placement
- Online payment
- Live order tracking
- Notifications

Excluded Features

- Food preparation management inside kitchen
- Bank internal transaction processing
- Restaurant inventory management beyond order quantity

1.3 Definitions, Acronyms, and Abbreviations

- OTP- One Time Password
- UI- User Interface

- API- Application Programming Interface
- GPS- Global Positioning System
- DB- Database
- Admin- System administrator
- HTTPS- Secure Hyper Text Transfer Protocol

1.4 References

- IEEE 29148 Software Requirement Standard
- Project Proposal Document
- REST API Documentation Standards
- Payment Gateway Integration Guide

1.5 Overview

This SRS describes system overview, functional requirements, interfaces, data requirements, system models, quality attributes, constraints, and acceptance criteria.

2. Overall Description

2.1 Product Perspective

The system is a client-server application consisting of:

- Mobile/Web Client (Users)
- Application Server (Backend APIs)
- Database Server
- External Services (Payment Gateway, Maps, SMS)

2.2 Product Functions

- Account registration and login
- Restaurant search and filtering
- Food ordering and cart management
- Online payment processing
- Delivery assignment
- Order tracking
- Notifications and history
- Admin management

2.3 User Classes and Characteristics

User	Skills	Responsibilities
Customer	Basic smartphone usage	Order food, track delivery
Restaurant Owner	Moderate technical knowledge	Accept/prepare orders
Delivery Agent	Mobile app usage	Pick & deliver orders
Admin	Technical knowledge	Manage users & restaurants

2.4 Operating Environment

- Android 8+, iOS 13+, Web browsers (Chrome, Edge)
- Backend: Linux server
- Database: MySQL/PostgreSQL
- Network: Internet (4G/5G/WiFi)

2.5 Design and Implementation Constraints

- Must support HTTPS communication
- Must integrate payment gateway API
- Mobile responsive UI required
- Database relational structure mandatory

2.6 User Documentation

- User guide manual
- Online help pages
- FAQs
- Video tutorials

2.7 Assumptions and Dependencies

- User has active internet connection
- GPS enabled for tracking
- External APIs available

3. Functional Requirements

3.1 System Features

Feature 1: User Authentication

Description: Allows users to register and login **Priority:** High **Inputs:** Email, password, OTP **Processing:** Validate credentials **Outputs:** Dashboard or error message

Feature 2: Restaurant Search

Description: Search nearby restaurants **Priority:** High **Inputs:** Location, filters **Processing:** Query database **Outputs:** Restaurant list

Feature 3: Order Placement

Description: Add items and place order **Priority:** High **Inputs:** Cart items **Processing:** Calculate total price **Outputs:** Order confirmation

Feature 4: Payment Processing

Description: Secure payment handling **Priority:** High **Inputs:** Payment details **Processing:** API transaction validation **Outputs:** Payment success/failure

Feature 5: Order Tracking

Description: Track order in real time **Priority:** Medium **Inputs:** Order ID **Processing:** GPS updates **Outputs:** Live map status

3.2 Use Case Specifications

Use Case: Place Order

Actors: Customer **Pre-conditions:** User logged in **Main Flow:**

- Select restaurant
- Add items to cart
- Choose payment method
- Confirm order

Alternate Flow: Payment failed **Post-condition:** Order stored in database

4. External Interface Requirements

4.1 User Interfaces

- Login screen
- Menu screen
- Cart screen
- Payment screen
- Tracking screen

Accessibility: readable fonts, color contrast

4.2 Hardware Interfaces

- GPS module
- Mobile camera (QR scanning)

4.3 Software Interfaces

- Payment gateway API
- Maps API
- SMS API

4.4 Communication Interfaces

- HTTPS protocol
- JSON data format

5. Non-Functional Requirements

5.1 Performance Requirements

- Response time < 3 seconds
- Support 5000 concurrent users

5.2 Security Requirements

- Password encryption

- Secure payment
- Role-based access

5.3 Reliability Requirements

- 99% uptime
- Automatic recovery

5.4 Usability Requirements

- Simple UI
- Consistent design

5.5 Maintainability

- Modular architecture
- Proper documentation

5.6 Scalability

- Cloud deployment
- Load balancing

5.7 Portability

- Runs on Android, iOS, Web

6. System Models (Lab 2 & Lab 3)

6.1 Use Case Diagrams

Users interact with system for ordering, tracking and managing food delivery.

6.2 Data Flow Diagrams (DFD)

Customer → System → Database → Restaurant → Delivery Agent

6.3 Sequence Diagrams

Customer → Server → Payment → Restaurant → Delivery Agent → Customer

7. Data Requirements

7.1 Data Description

- User ID (integer)
- Order ID (integer)
- Price (decimal)
- Status (string)

Validation: email format, positive price

7.2 Database Requirements

Tables:

- Users
- Restaurants
- Orders
- Payments
- Delivery Agents

Primary keys and foreign keys defined

8. Quality Attributes

- Correctness
- Efficiency
- Flexibility
- Reusability
- Testability

9. Constraints and Risks

- API downtime risk
- Network failures
- Security attacks

10. Acceptance Criteria

- All functional requirements verified
- Payment successful test cases passed
- Performance benchmarks achieved

11. Appendices

Glossary

User – customer using application Order – food purchase request Restaurant – food provider.

Note:

- Students must prepare a complete SRS document of the assigned topic.
- Topic of the SRS should not be repeated.
- Laboratory manual is only for reference and guiding purpose. Do Not Copy.
- Students should take screenshots of the steps, attach and label them with one line description (if applicable).

Conclusion:

Hence, I have Prepared a complete Software Requirement Specification (SRS) document for an assigned system (Online Food Delivery Application).

Laboratory Exercise No. 2

Aim: Draw a Use Case, Activity & Data Flow Diagram for an assigned system (e.g., Library Management System/Online Food Delivery Application) using draw.io tool.

Theory:

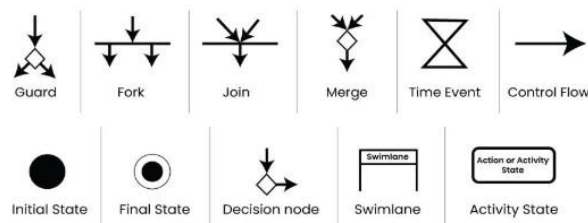
Use Case Diagram:

- A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals. It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.
- Actors are external entities that interact with the system. These can include users, other systems, or hardware devices. In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes. Proper identification and understanding of actors are crucial for accurately Modeling system behaviour.
- Use cases are like scenes in the play. They represent specific things your system can do. In the online shopping system, examples of use cases could be "Place Order," "Track Delivery," or "Update Product Information". Use cases are represented by ovals.
- The system boundary is a visual representation of the scope or limits of the system you are Modeling. It defines what is inside the system and what is outside. The boundary helps to establish a clear distinction between the elements that are part of the system and those that are external to it. The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system.

Activity Diagram:

- Activity diagrams show the steps involved in how a system works, helping us understand the flow of control. They display the order in which activities happen and whether they occur one after the other (sequential) or at the same time (concurrent). These diagrams help explain what triggers certain actions or events in a system.
- They depict how different actions are connected and how a system moves from one state to another.

Notations:



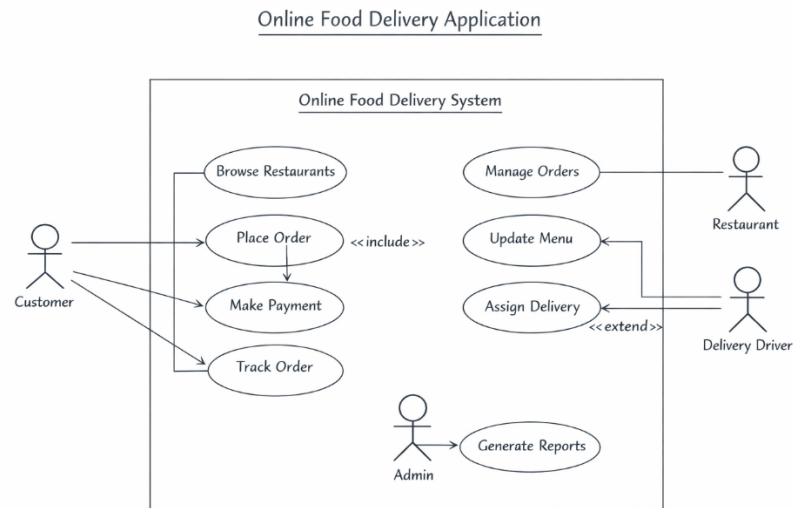
Data Flow Diagram:

- A Data Flow Diagram (DFD) is a graphical tool used to represent how data moves through a system. It shows data inputs, outputs, data stores, and the processes that transform the data. DFDs provide a high-level overview of system functionality and are widely used in structured analysis due to their simplicity and clarity for both technical and non-technical users.
- Levels of Data Flow Diagram
- Level 0 DFD (Context Diagram): Represents the entire system as one single process, showing its interaction with external entities. It highlights major inputs and outputs without internal details.
- Level 1 DFD: Breaks the Level 0 process into major subprocesses. Shows internal data flows and data stores.
- Level 2 and Beyond: Further decomposes Level 1 subprocesses for a detailed view of specific functional areas. Useful for large or complex systems requiring deeper analysis.
- The Logical Data Flow Diagram mainly focuses on the system process. It illustrates how data flows in the system. Logical Data Flow Diagram (DFD) mainly focuses on high level processes and data flow without diving deep into technical implementation details.
- Physical data flow diagram shows how the data flow is implemented in the system. In the Physical Data Flow Diagram (DFD)

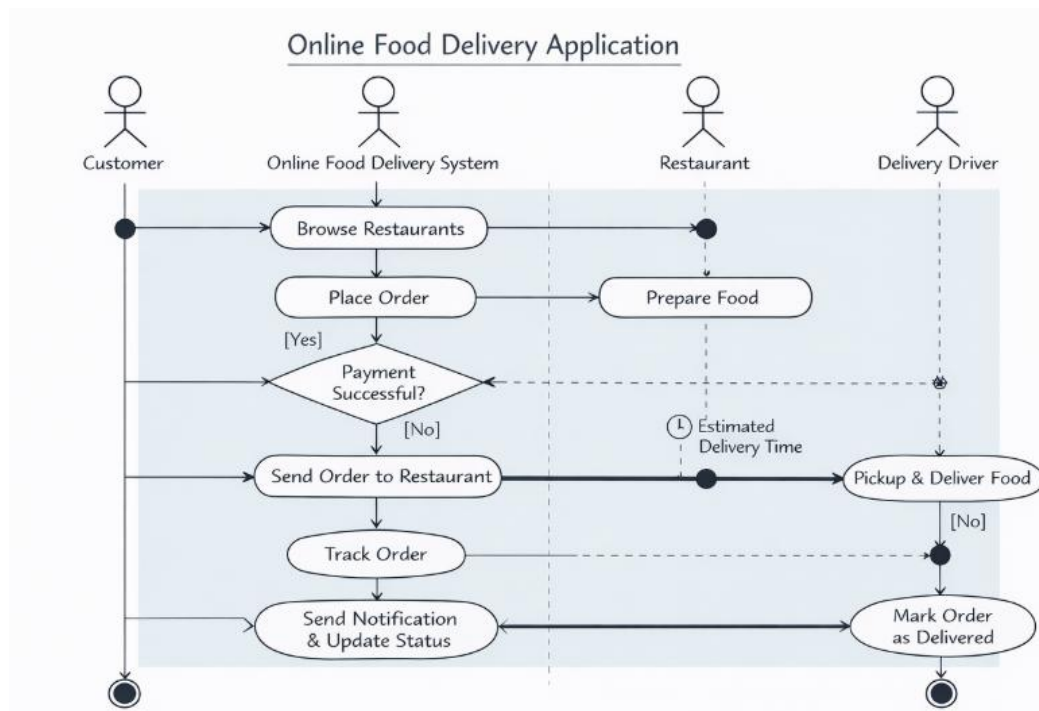
Performance:

Sample Diagrams:

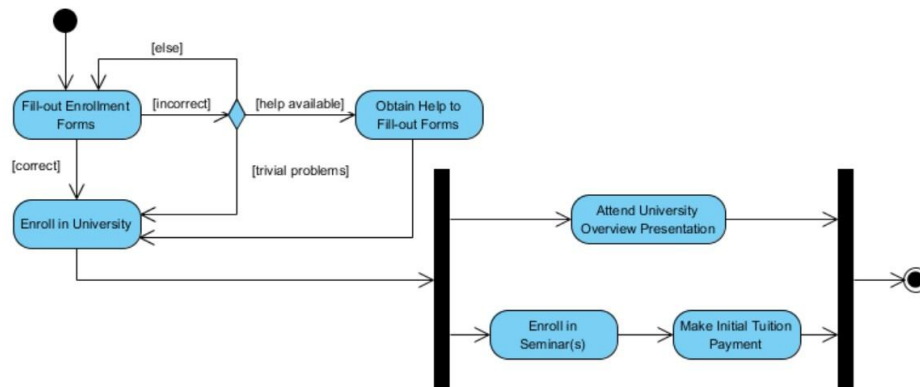
Use Case Diagram:



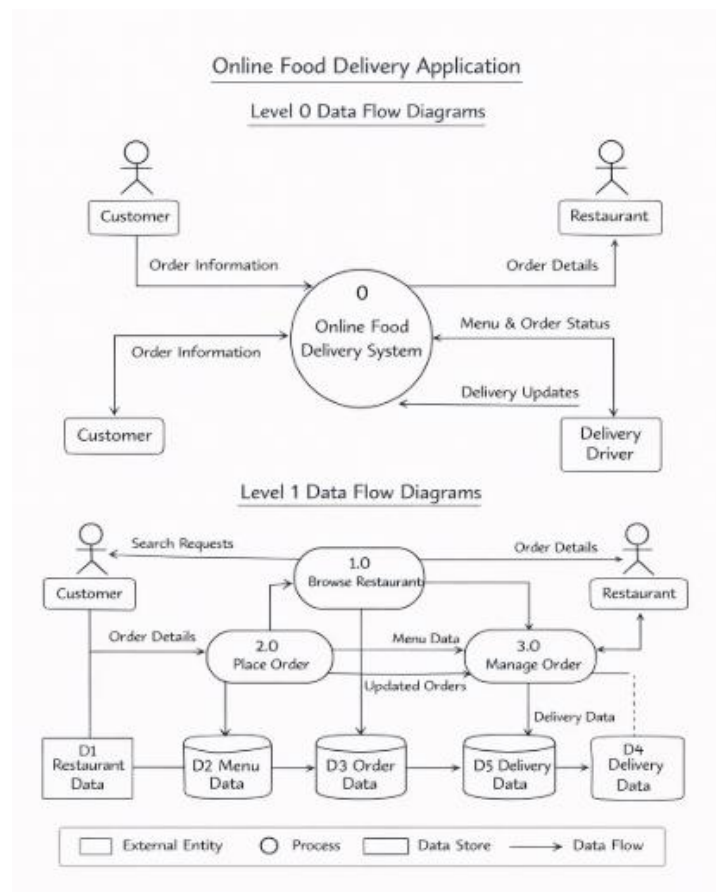
Sample Activity Diagram:



Sample Activity Diagram: Enrollment



Sample Data Flow Diagram:



Note:

- Students should use proper notations, actors and show proper relationships while drawing the use case diagram, activity diagram and data flow diagram.
- Students should take screenshots of the diagrams or image file, attach and label them with one line description.
- Students should draw and explain all the diagrams of the assigned topic in brief.
- Laboratory manual is only for reference and guiding purpose. Do Not Copy.

Conclusion:

Hence, I have Drawn and explained Use Case, Activity & Data Flow Diagram for an assigned system (Online Food Delivery Application).

Laboratory Exercise No. 3

Aim: Draw a Use Class diagram and Sequence diagram for an assigned system (e.g., Library Management System/Online Food Delivery Application) using draw.io tool.

Theory:

A Sequence Diagram is a key component of Unified Modeling Language (UML) used to visualize the interaction between objects in a sequential order. They focus on the order and timing of messages or events exchanged between different system elements.

A sequence diagram shows how objects interact with each other over time in a system. It helps visualize the order of messages and operations between objects.

- Illustrates interactions and message flows between objects
- Shows the sequence of operations over time
- Helps understand use cases, design system architecture, and document processes

Key Components of a Sequence Diagram

- **Actors/Objects:**

Represented by vertical boxes at the top, showing participants (e.g., users, subsystems)

- **Lifelines:** Vertical dashed lines representing the existence of an object over time.
- **Activation Boxes:** Thin rectangles on lifelines indicating when an object is actively performing an operation.
- **Messages:** Horizontal arrows indicating communication between objects.
 - **Synchronous:** Solid line filled arrowhead (sender waits for response).
 - **Asynchronous:** Solid line, open arrowhead (sender does not wait).
 - **Return:** Dashed line, open arrowhead.

A UML class diagram shows the structure of a system by displaying its classes, their attributes, methods, and the relationships between them. It helps the team understand how the system is organized and how components interact.

- Represents classes, attributes, methods, and relationships
- Helps communicate and document the software structure
- Makes it easier for developers and designers to understand the system

UML Class Notation

1. **Class Name:** The name of the class is typically written in the top compartment of the class box and is centered and bold.
2. **Attributes:** Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class

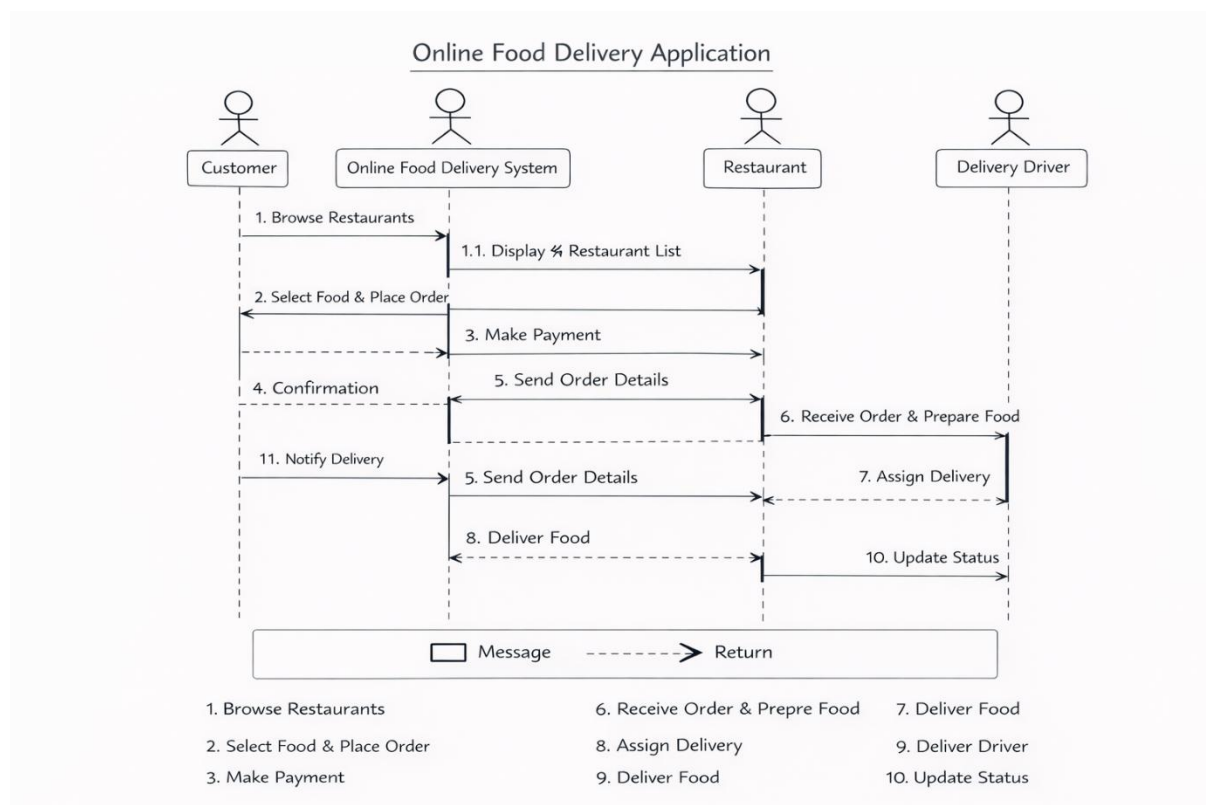
box and often include the visibility (e.g., public, private) and the data type of each attribute.

3. **Methods:** Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.
4. **Visibility Notation:** Visibility notations indicate the access level of attributes and methods. Common visibility notations include:
 - + for public (visible to all classes)
 - - for private (visible only within the class)
 - # for protected (visible to subclasses)
 - ~ for package or default visibility (visible to classes in the same package)

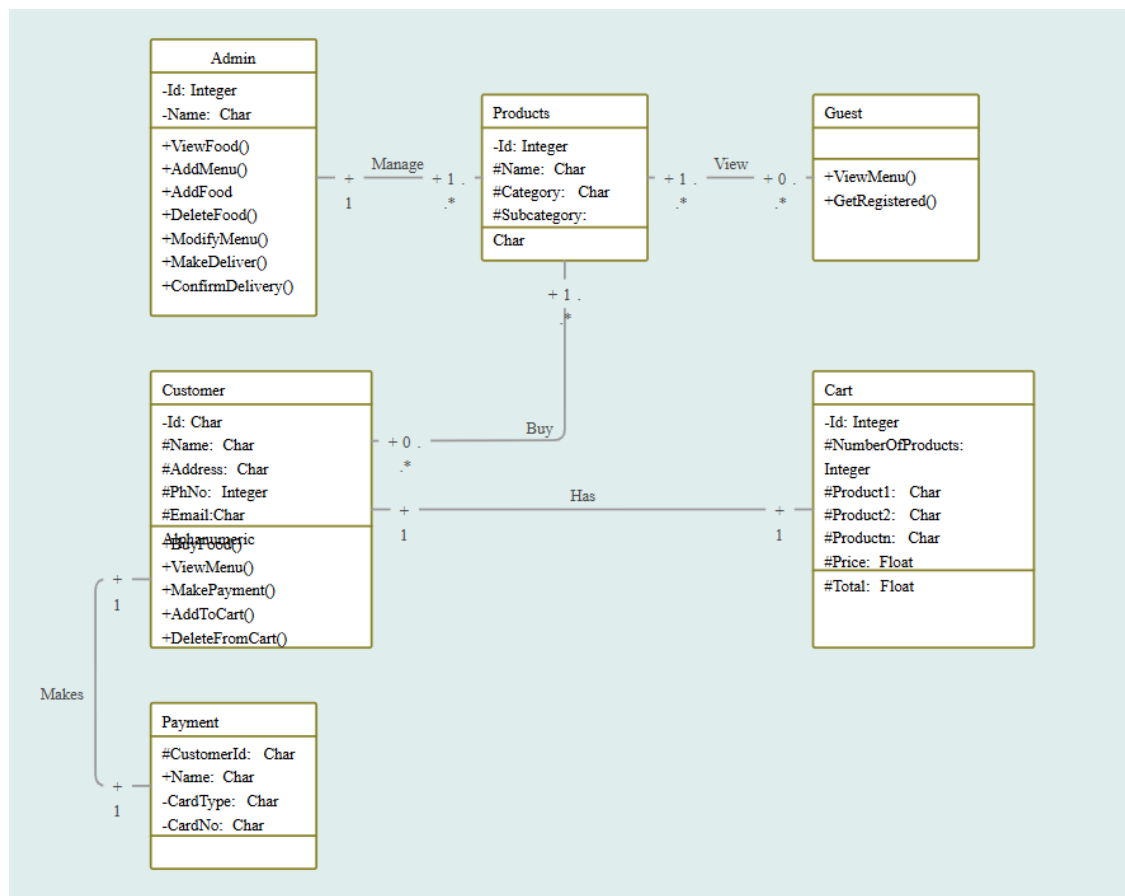
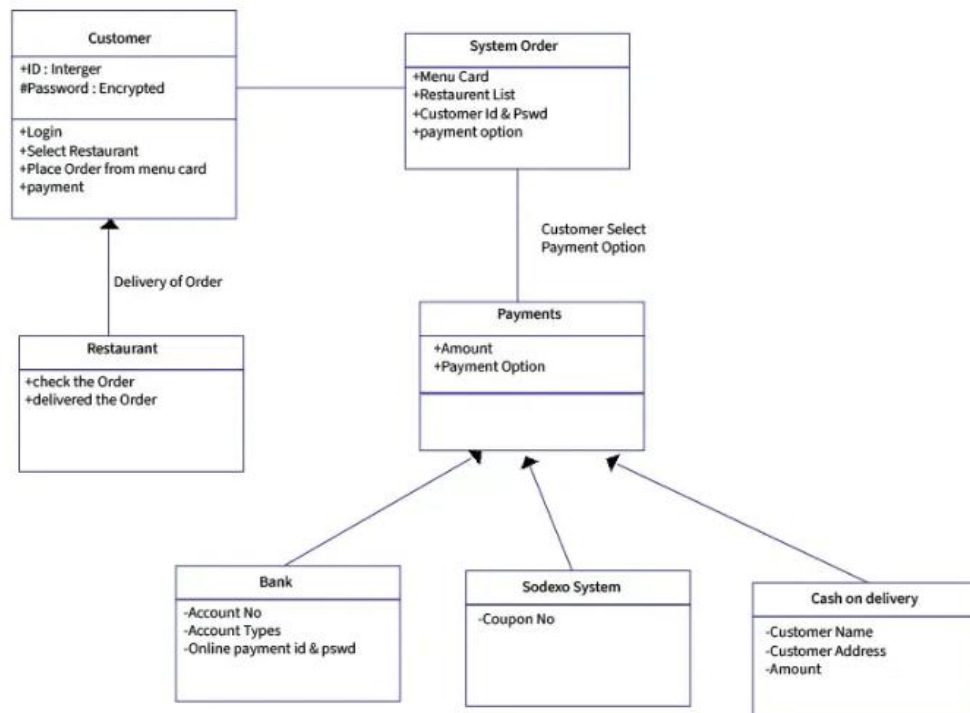
Performance:

Sample Diagrams:

Sequence Diagram:



Class Diagram:



Note:

- Students should use proper notations, actors and show proper relationships while drawing the sequence diagram and class diagram.
- Students should take screenshots of the diagrams or image file, attach and label them with one line description.
- Students should draw and explain all the diagrams of the assigned topic in brief.
- Laboratory manual is only for reference and guiding purpose. Do Not Copy.

Conclusion:

Hence, I have Drawn and explained Sequence Diagram & Class Diagram for an assigned system (Online Food Delivery Application).

Laboratory Exercise No. 4

Aim: To perform repository creation, branching, merging, pull request handling, and issue tracking using GitHub.

Theory:

Git is a free, open-source distributed version control system (DVCS) designed to manage and track changes in source code and other files during software development. It allows multiple developers to work collaboratively on the same project, tracking every modification and facilitating the seamless integration of changes.

A Version Control System (VCS) is a tool that helps developers track and manage changes in source code over time. It allows multiple developers to collaborate on a project without overwriting each other's work.

One of the most widely used version control systems is Git, which was created by Linus Torvalds in 2005.

Features of Git

- Distributed system
- Fast performance
- Secure version tracking
- Supports branching and merging
- Enables team collaboration

Git is commonly used with platforms like GitHub, GitLab, and Bitbucket.

Git Architecture

Git has three main areas:

1. Working Directory

- The folder where project files are stored and edited.

2. Staging Area

- Temporary area where changes are prepared before committing.

3. Repository

- Stores the complete project history.

Flow

Working Directory → Staging Area → Git Repository

5. Key Git Concepts

Repository (Repo)

A repository is a storage location for project files and their version history.

Commit

- A commit saves the current changes to the repository.
- Example command:
- `git commit -m "Added login functionality"`

Each commit includes:

- Author
- Date
- Message
- Changes made

Branch

A **branch** allows developers to work on different features independently.

Example:

```
git branch feature-login
```

Benefits

- Parallel development
- Isolated feature implementation

Merge

- Merging combines changes from one branch into another.
- Example:
- `git merge feature-login`

Clone

- Cloning creates a copy of a remote repository.
- Example:
- `git clone repository_url`

Pull

- Pull retrieves the latest changes from a remote repository.
- Example:
- `git pull origin main`

Push

- Push uploads local commits to the remote repository.
- Example:
- `git push origin main`

6. Basic Git Workflow

The typical Git workflow involves the following steps:

Step 1: Initialize repository

```
git init
```

Step 2: Add files

```
git add filename
```

Step 3: Commit changes

```
git commit -m "Initial commit"
```

Step 4: Push to remote repository

```
git push origin main
```

7. Common Git Commands

Command Description

git init	Create new repository
git clone	Copy repository
git add	Add files to staging area
git commit	Save changes
git push	Upload changes
git pull	Download updates
git branch	Create branch
git merge	Combine branches
git status	Show repository status
git log	View commit history

Tools Required

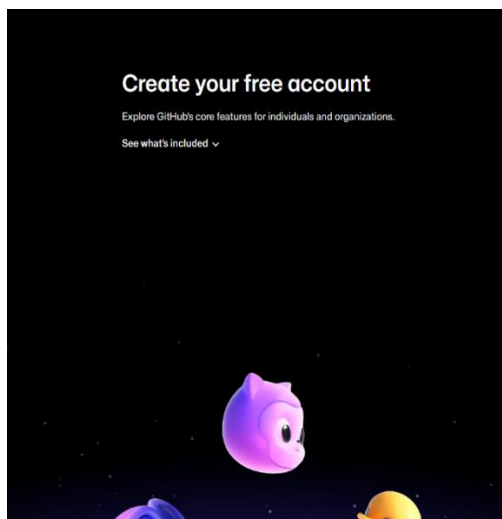
- Git Bash
- GitHub Account
- VS Code

Performance:

PART 1 — Creating Repository

Step 1: Create GitHub Account

- Go to <https://github.com>
- Click **Sign Up**
- Complete registration and login



Already have an account? [Sign in](#)

Sign up for GitHub

Continue with Google

Continue with Apple

or

Email^{*}

Email

Password^{*}

Password

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username^{*}

Username

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

Your Country/Region^{*}

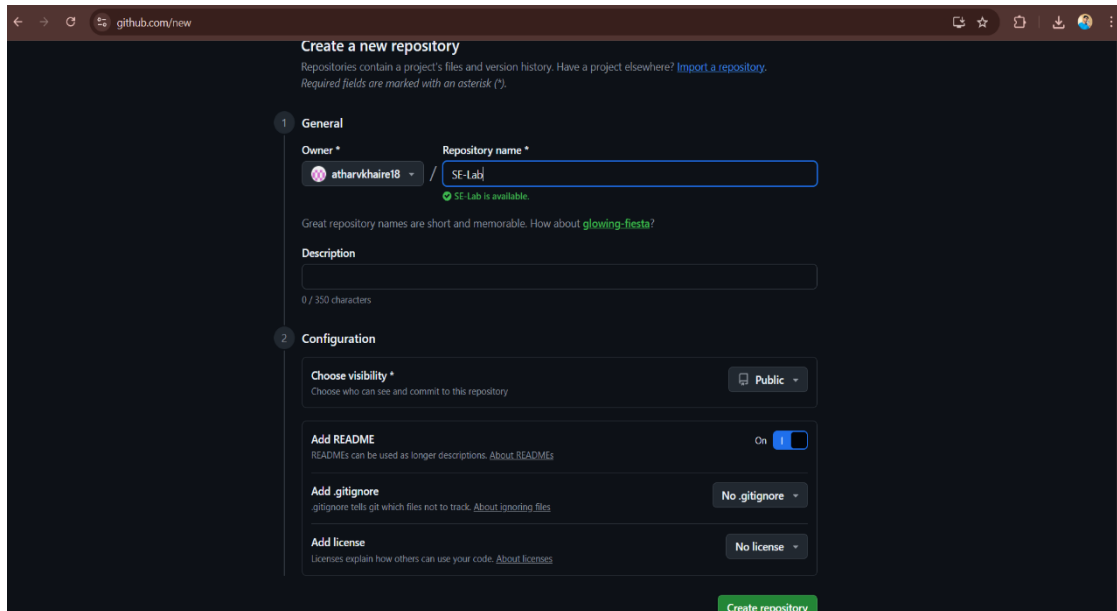
India

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Email preferences

Step 2: Create Repository

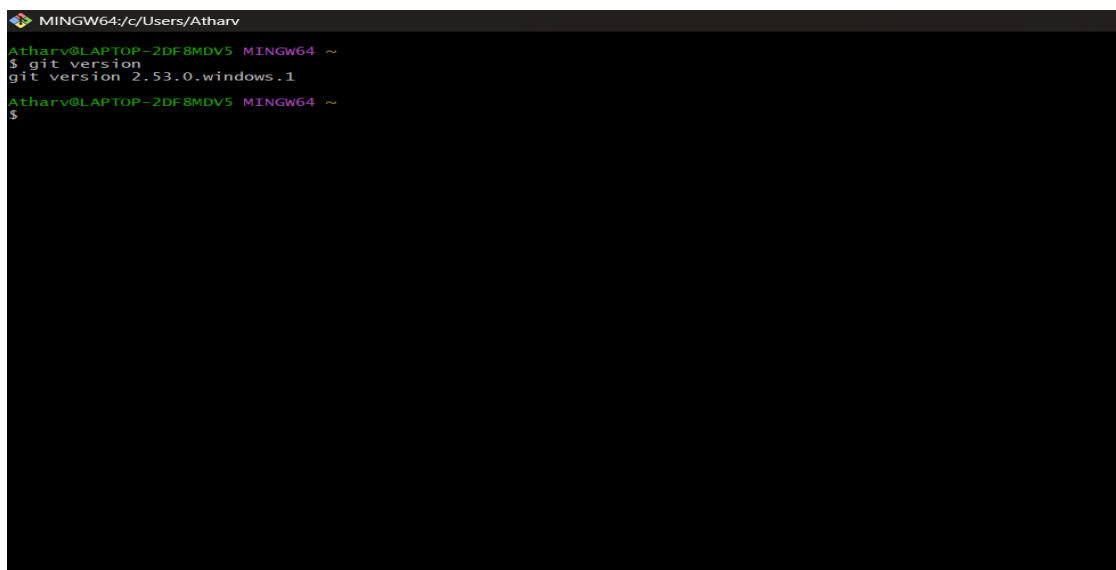
1. Click **New Repository**
2. Repository Name: SE-Lab
3. Select **Public**
4. Tick **Add README file**
5. Click **Create Repository**



PART 2 — Clone Repository

Step 1: Open Git Bash

- Right-click in folder → Click **Git Bash Here**
- Check Git version:
- `git --version`



Step 2: Clone Repository

- `git clone https://github.com/softengg/SE-Lab.git`
- `cd SE-Lab`

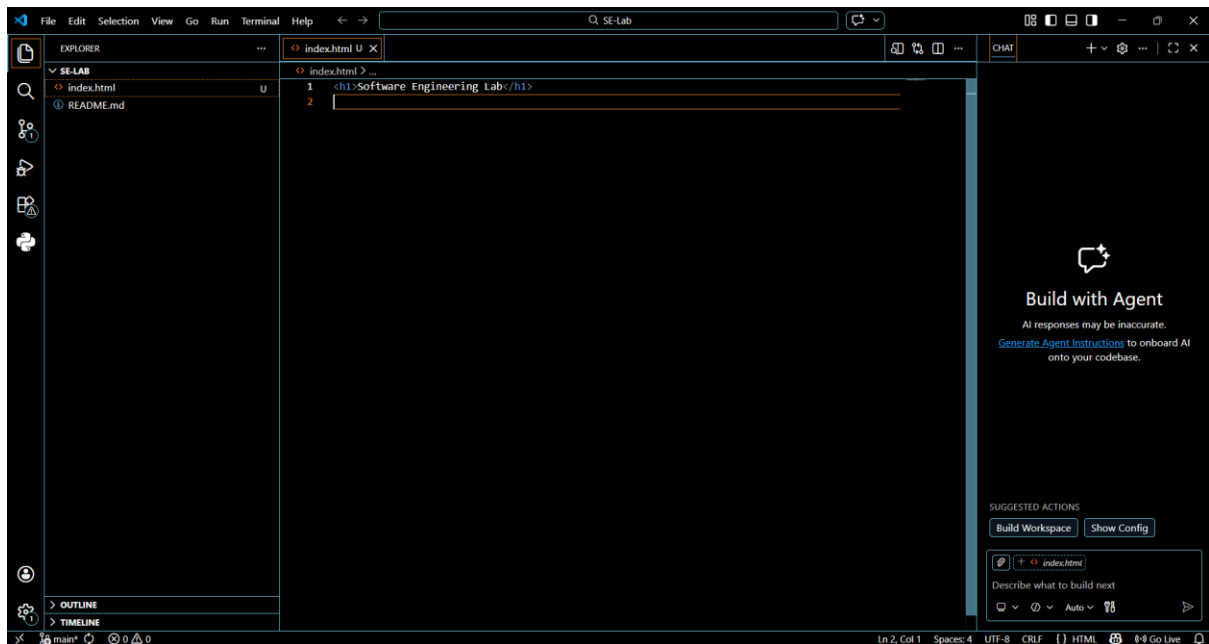
```
MINGW64/c/Users/Atharv
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~
$ git version
git version 2.53.0.windows.1
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~
$ git clone https://github.com/atharvkhaire18/SE-Lab.git
Cloning into 'SE-Lab'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~
$ |
```

PART 3 — Add & Commit Files

Step 1: Create index.html

- `touch index.html`
- Add content:
- `<h1>Software Engineering Lab</h1>`

```
MINGW64/c/Users/Atharv/SE-Lab
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~
$ git version
git version 2.53.0.windows.1
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~
$ git clone https://github.com/atharvkhaire18/SE-Lab.git
Cloning into 'SE-Lab'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~
$ cd SE-Lab
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ touch index.html
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ explorer .
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ |
```

Step 2: Add & Commit

- git status
- git add index.html
- git commit -m "Added homepage"
- git push origin main

```
MINGW64/c/Users/Atharv/SE-Lab
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ ls
README.md  index.html

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ ^[[200~code .
bash: $'\E[200~code': command not found

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ code .

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html

nothing added to commit but untracked files present (use "git add" to track)

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git add index.html

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git commit -m "Added homepage"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

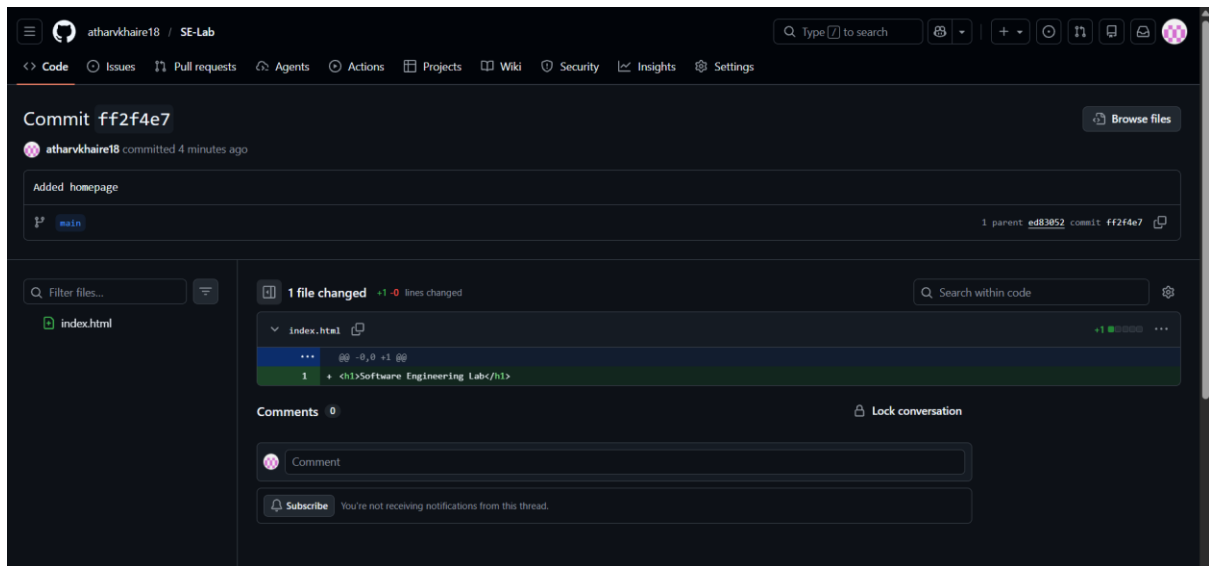
fatal: unable to auto-detect email address (got 'Atharv@LAPTOP-2DF8MDV5.(none)')
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git config --global user.email "atharvkhair18@gmail.com"

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git config --global user.name "atharv khair"

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git add index.html

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git commit -m "Added homepage"
[main ff2f4e7] Added homepage
 1 file changed, 1 insertion(+)
 create mode 100644 index.html

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ |
```



PART 4 — Branching

Branching allows developers to work independently without affecting the main code.

Step 1: Create New Branch

git checkout -b feature-login

```
MINGW64/C/Users/Atharv/SE-Lab
fatal: unable to auto-detect email address (got 'Atharv@LAPTOP-2DF8MDV5.(none)')

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git config --global user.email "atharvkhaire18@gmail.com"

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git config --global user.name "atharv khaire"

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git add index.html

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git commit -m "Added homepage"
[main ff2f4e7] Added homepage
1 file changed, 1 insertion(+)
create mode 100644 index.html

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git push origin main
info: please complete authentication in your browser...
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 310 bytes | 310.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/atharvkhaire18/SE-Lab.git
   ed83052..ff2f4e7  main -> main

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (main)
$ git checkout -b feature-login
Switched to a new branch 'feature-login'

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (feature-login)
$ git branch
* feature-login
  main

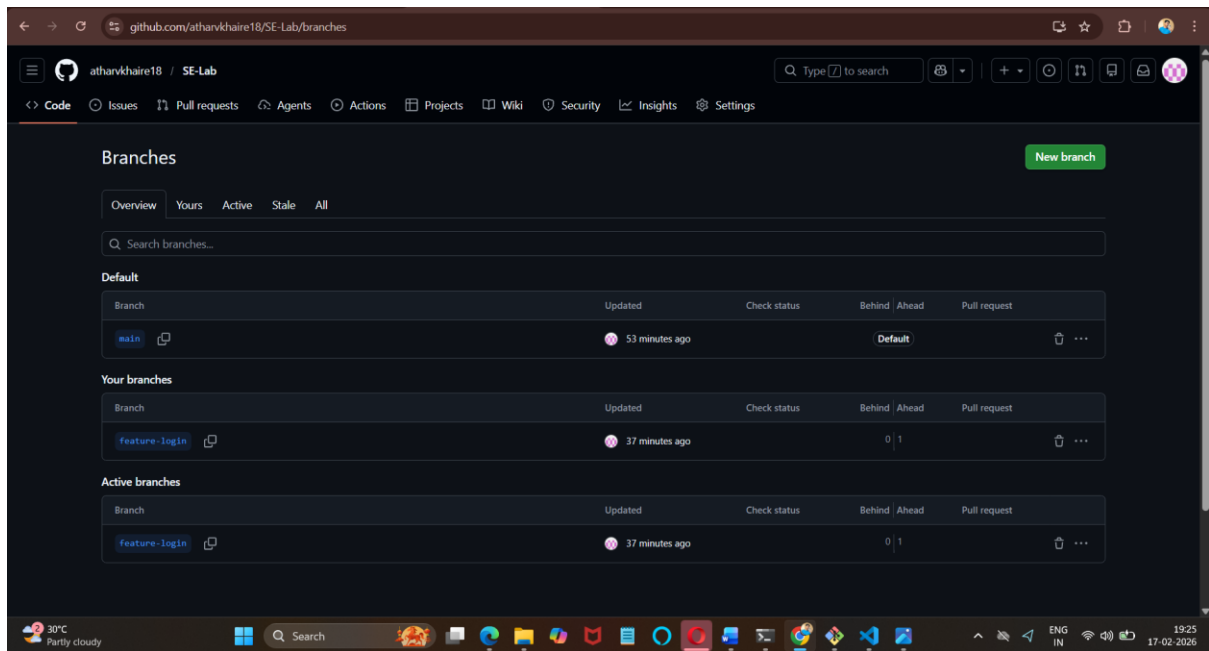
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (feature-login)
$ git checkout -b feature-login
fatal: a branch named 'feature-login' already exists

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (feature-login)
$ git checkout feature-login
Already on 'feature-login'

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (feature-login)
$ touch login.html

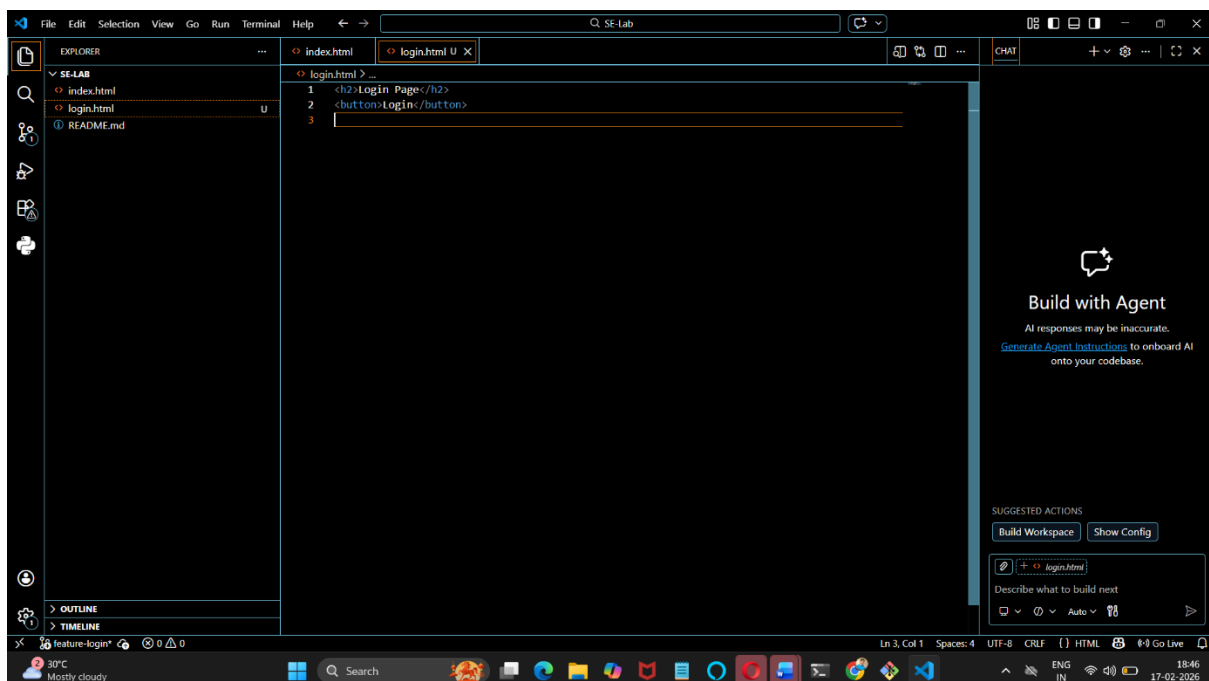
Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (feature-login)
$ git add .
$ git commit -m "Added login page"
[feature-login 831b8b7] Added login page
1 file changed, 2 insertions(+)
create mode 100644 login.html

Atharv@LAPTOP-2DF8MDV5 MINGW64 ~/SE-Lab (feature-login)
$
```



Step 2: Modify Code

- Create new file:
- touch login.html
- Add content:
- `<h2>Login Page</h2>`
- `<button>Login</button>`

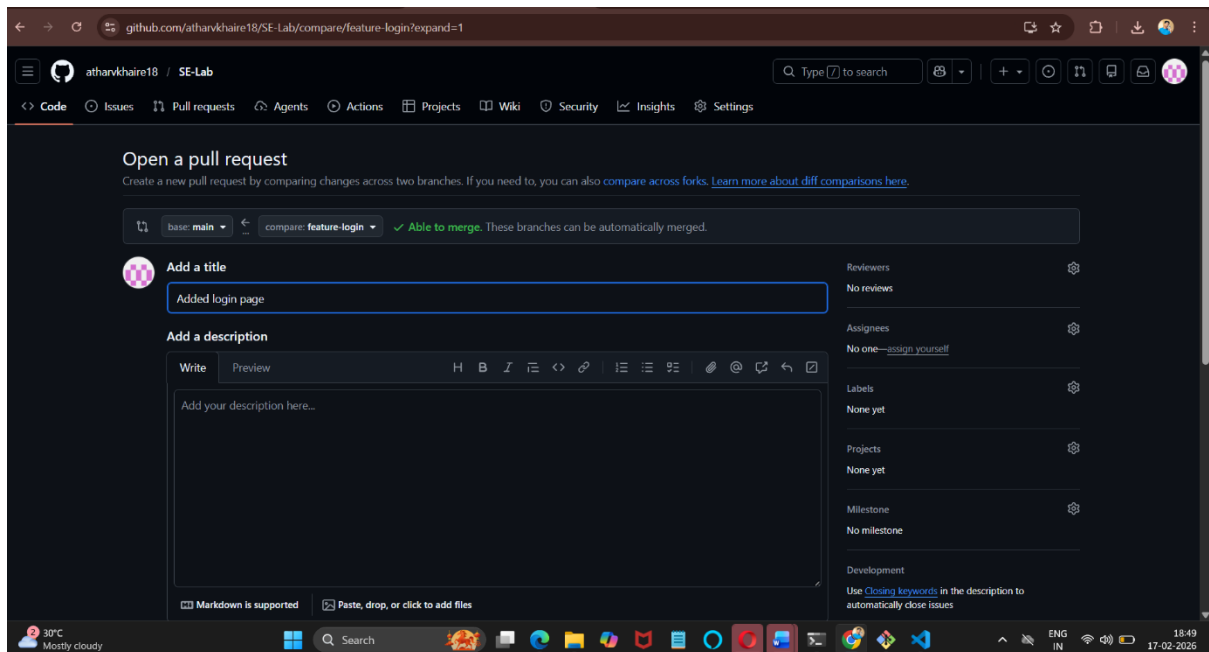


PART 5 — Pull Request (PR)

A Pull Request is used to merge changes from one branch into another.

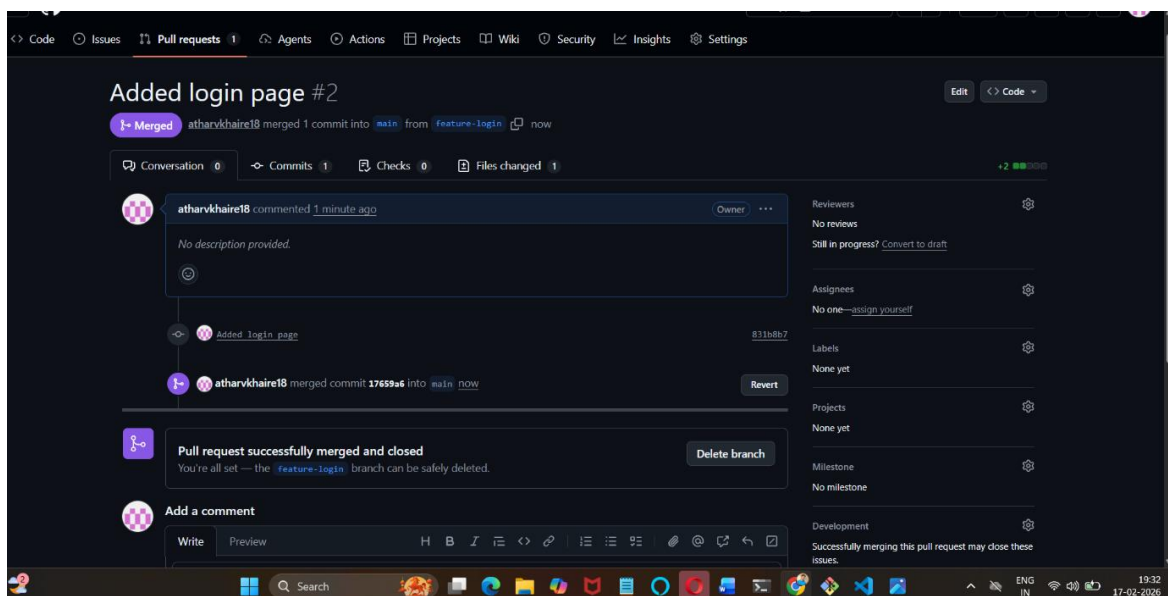
Steps on GitHub Website

1. Open repository
2. Click **Compare & Pull Request**
3. Add Title: Added login page
4. Add Description
5. Click **Create Pull Request**



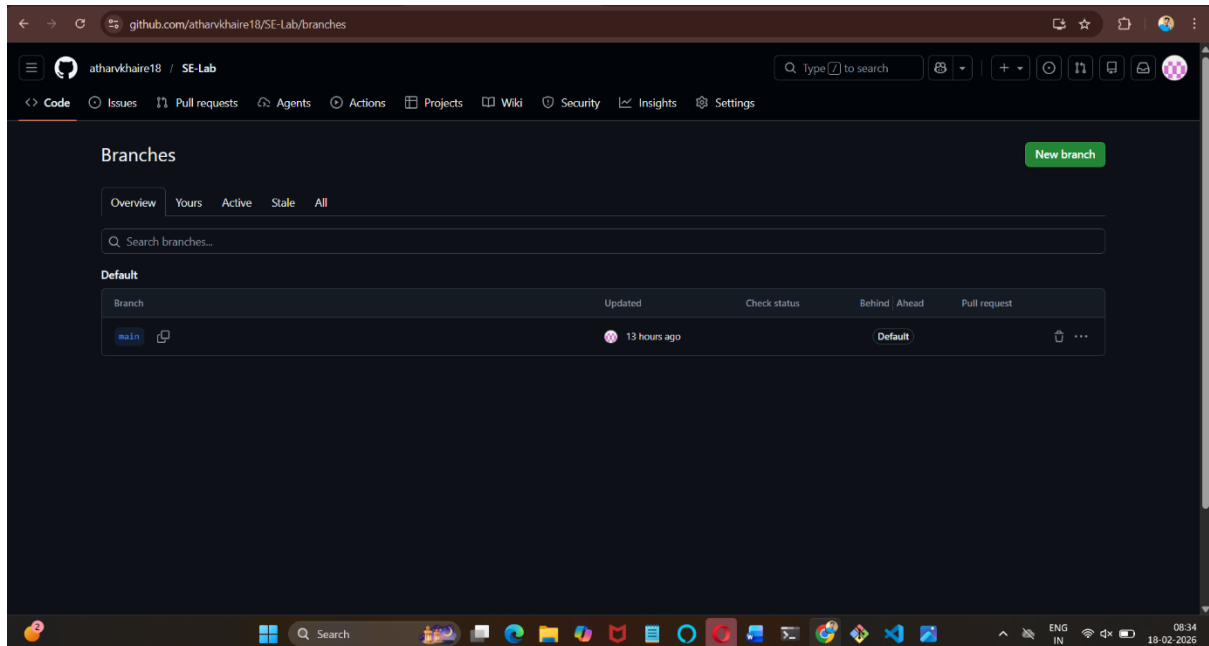
PART 6 — Merge Branch

1. Click **Merge Pull Request**
2. Click **Confirm Merge**



PART 7 — Update Local Repository

- git checkout main
- git pull origin main
- Optional: Delete branch
- git branch -d feature-login
- git push origin --delete feature-login



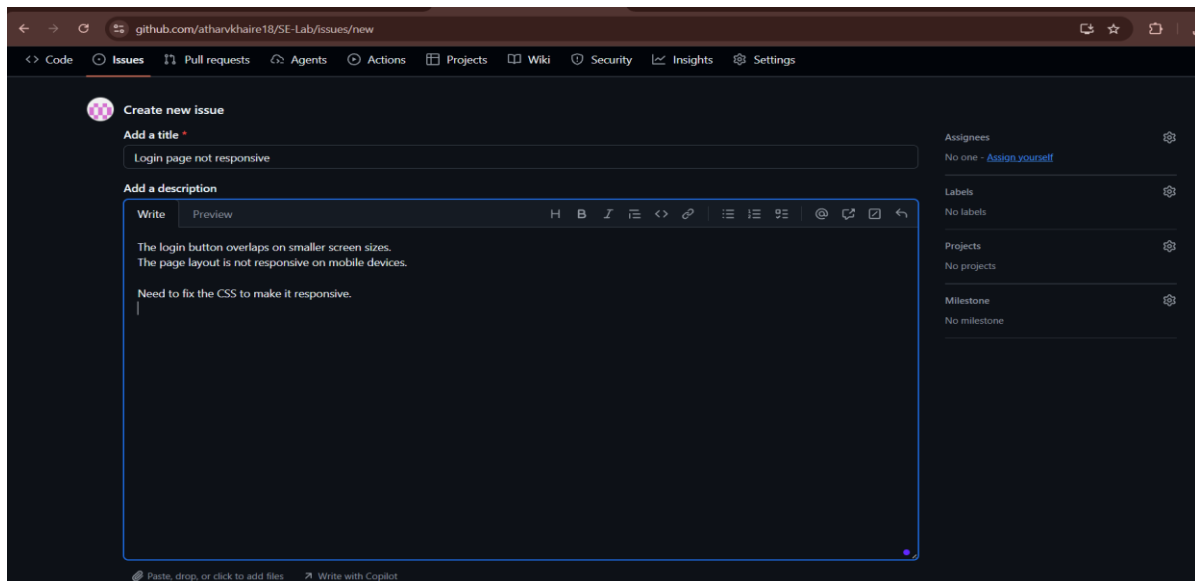
PART 8 — Issue Tracking

Used to track bugs, tasks, and enhancements.

Step 1: Create Issue

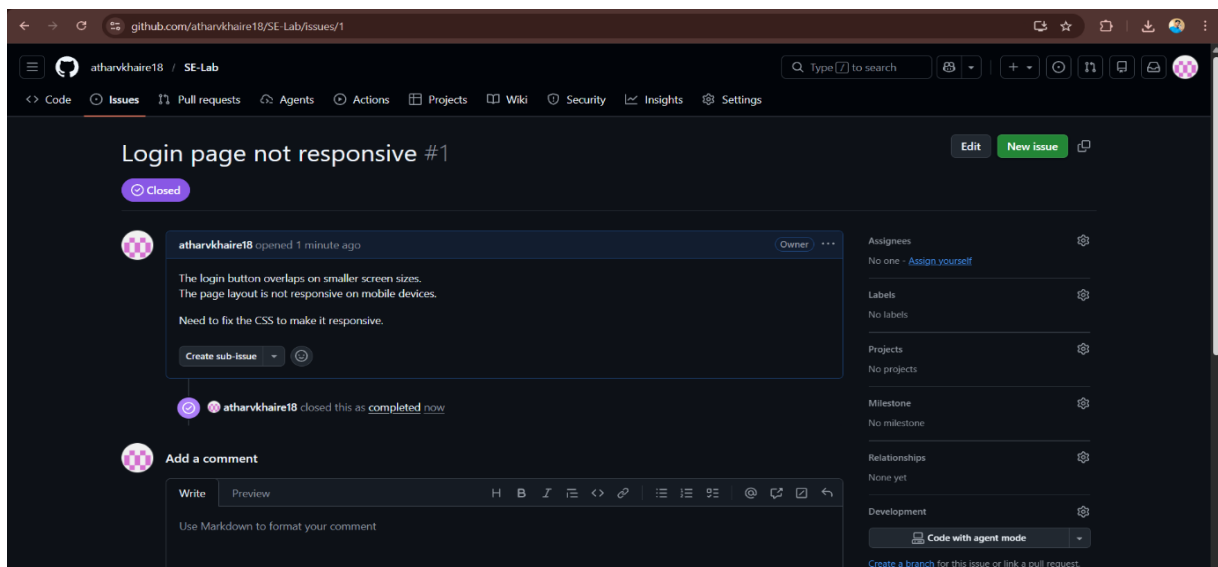
- Go to Repository → Issues → New Issue
- Title:
- Login page not responsive
- Description:
- The login button overlaps on mobile screen.
- Need to fix layout issue.

Click **Submit new issue**



Step 2: Close Issue

- After fixing issue:
- `git commit -m "Fixed login responsiveness #1"`
- `git push`
- GitHub automatically closes issue after merge.



Note:

- Laboratory manual is only for reference and guiding purpose. Do Not Copy.
- Students should take screenshots of the steps, attach and label them with one line description.

Conclusion:

Hence, I have performed, repository creation, branching, merging, pull request handling, and issue tracking using GitHub.